EE460N_LAB5; Author: Andrew Yin

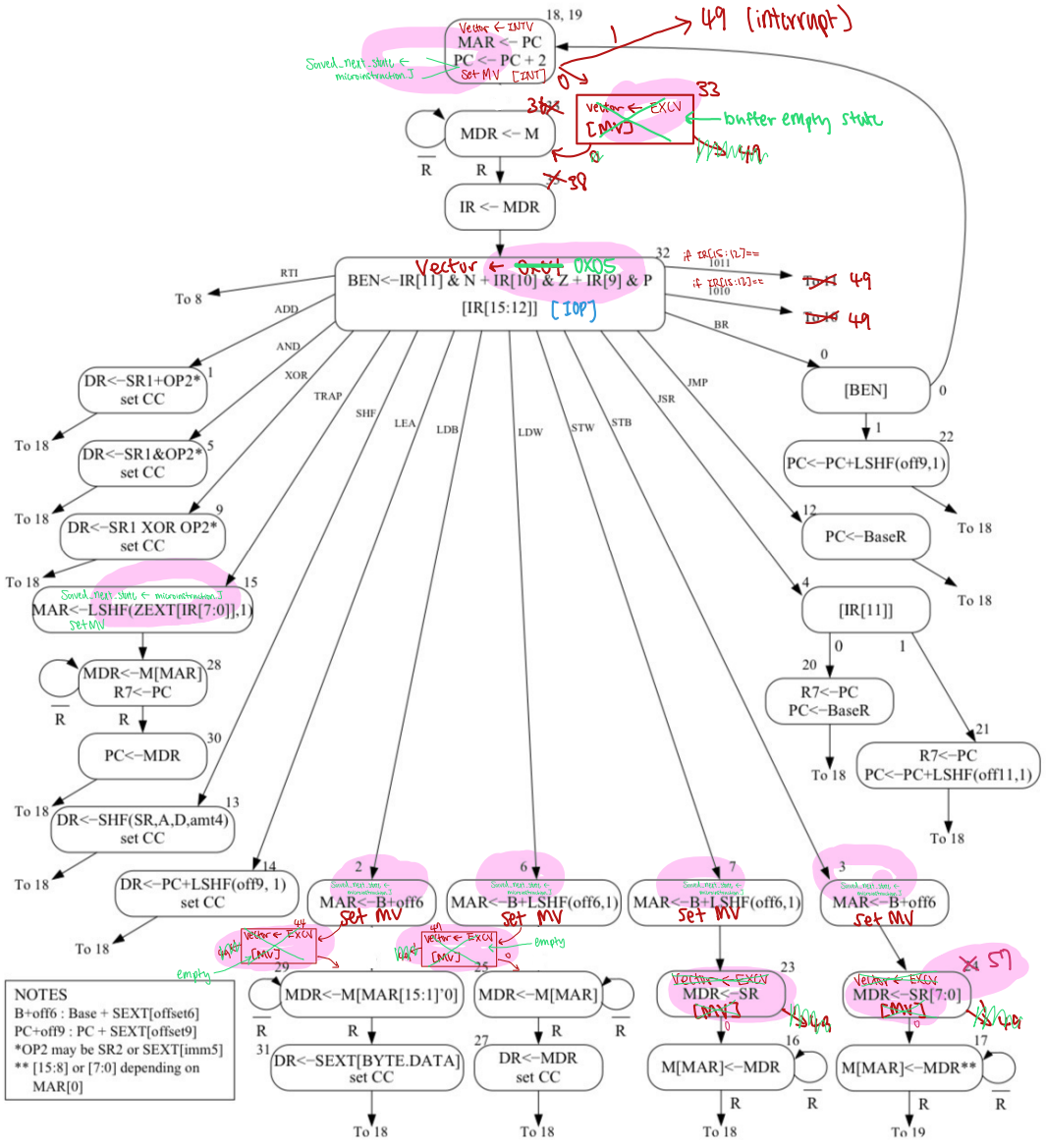
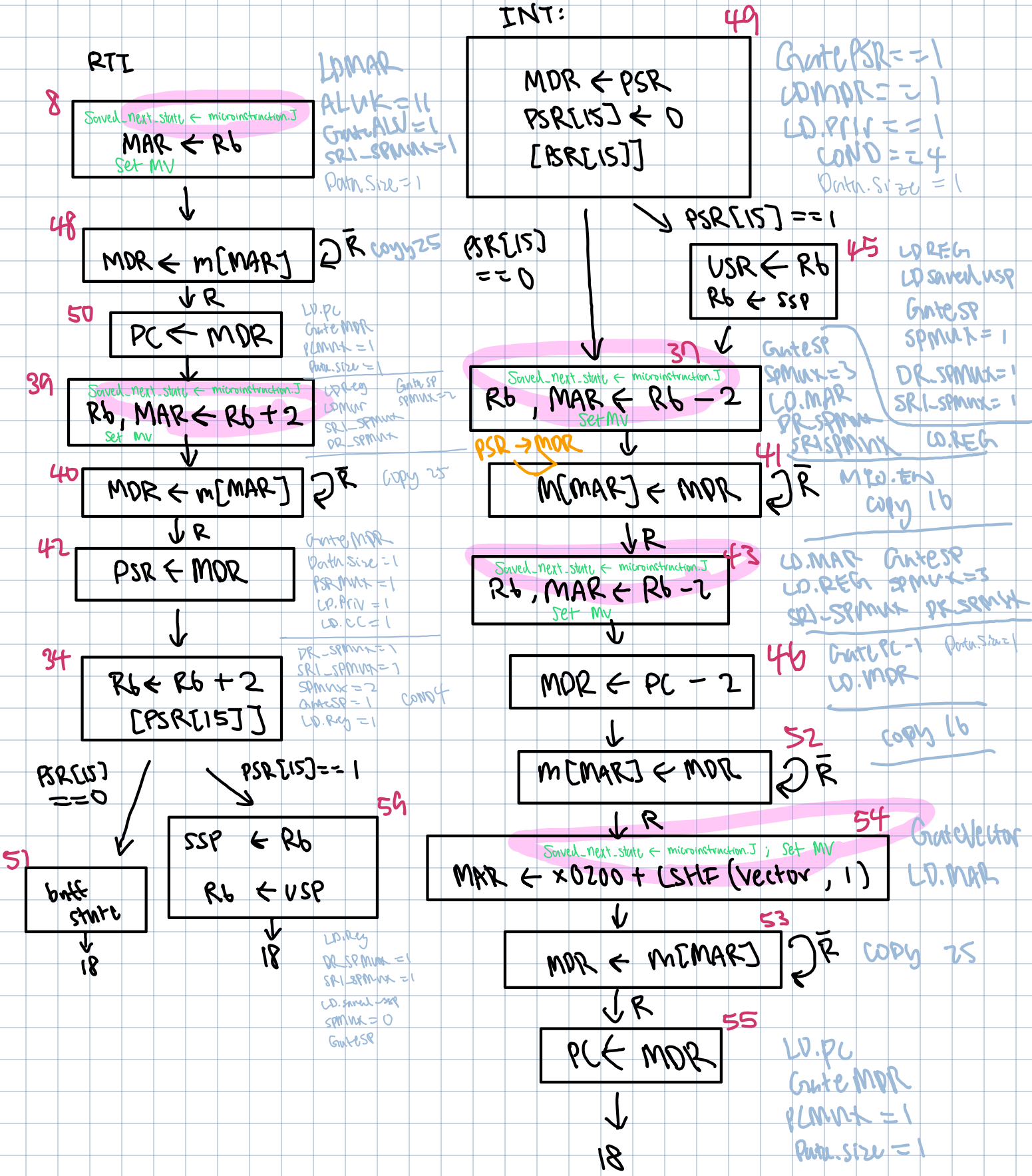


Figure C.2: A state machine for the LC-3b

- Total number of states: 59

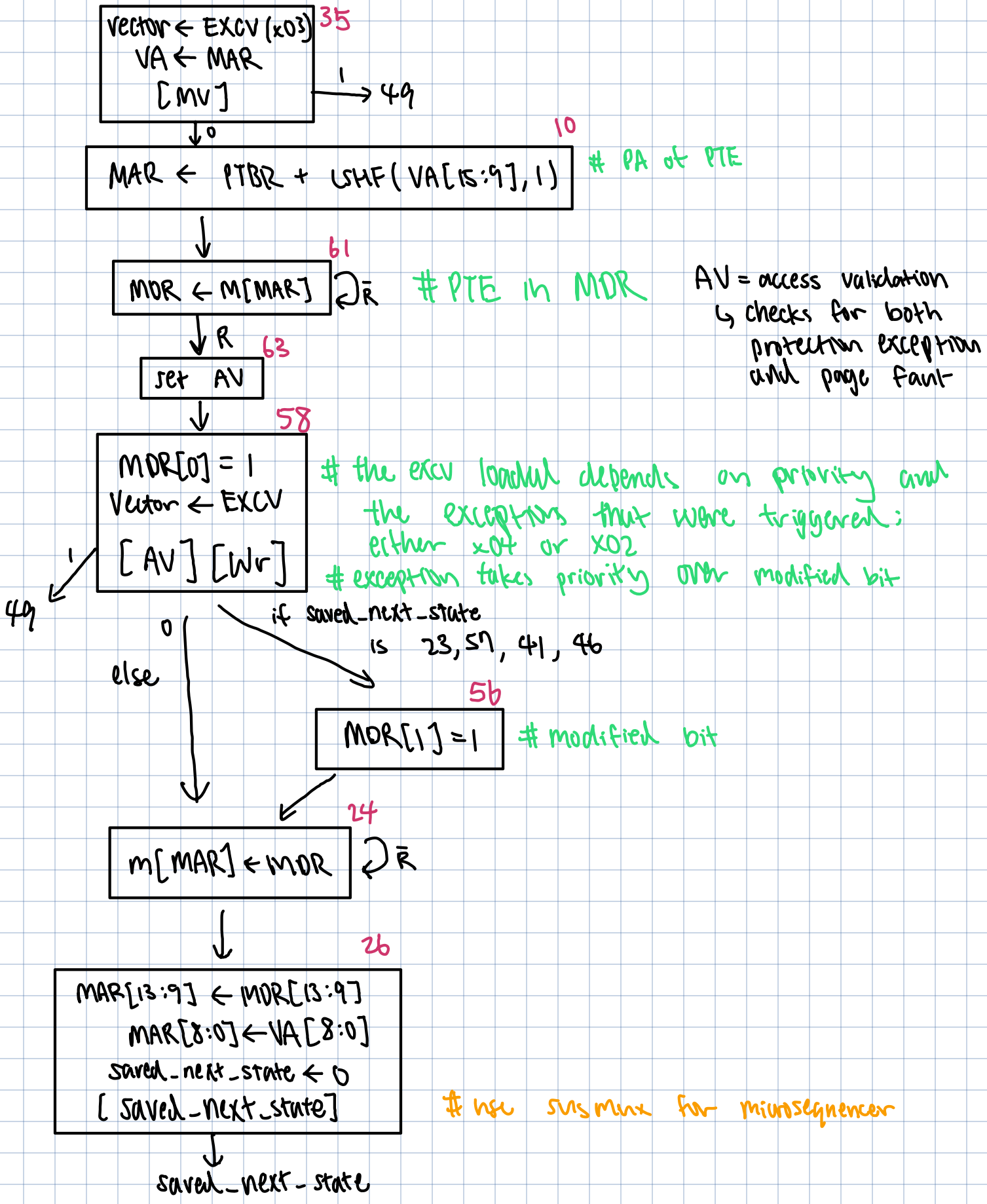
New states or changes to existing states (in red highlight, green texts above and in next 2 pages):

- 18, 19, 15, 2, 6, 7, 3, 8, 39, 37, 43, 54 - these states load virtual addresses into the MAR, which means that the next state should be to do translation. In these states, I made sure to save the next states (indicated in the J field of the micro instruction) into a register.
- 33, 44, 47, 23, 57 - I removed the parts that load a number into the vector register and removed the checking for memory violation for these states, because i could just check it and load into vector register in the first state of translation. This is required because unaligned access needs to be checked before physical address translation.
- 32 - diff exception vector for unknown opcode because diff specifications by the lab doc
- 35, 10, 61, 63, 58, 56, 24, 26 - the translation states used for translating from virtual to physical memory. This is done by first finding the PTE of a corresponding virtual page by adding the PTE with the VPN, then using the TPE to find the PFN of the page, and using the offset found in the virtual address to find the physical address of the corresponding virtual address, which is then pt back into the MAR in the last state of the sequence. Throughout the process, AV (access violation exceptions) are checked, and modified bit, reference bit of the PTE is changed. Saved_next_state register is used to return back to the state in the original sequence without translation.


RTI
8
Saved_next_state ← microinstruction.J
MAR ← R6
Set MV
LD.MAR
ALUK = 11
GateALU = 1
SR1_SPMUX = 1
Data.Size = 1
48
MDR ← m[MAR]
R
copy 25
R
50
PC ← MDR
LD.PC
GateMDR
PCMUX = 1
Data.size = 1
39
Saved_next_state ← microinstruction.J
R6, MAR ← R6 + 2
Set MV
LD.Reg
LD.MAR
SR1_SPMUX
DR_SPMUX
GateSP
SPMUX = 2
40
MDR ← m[MAR]
R
copy 25
42
PSR ← MDR
GateMDR
Data.Size = 1
PSRMUX = 1
LD.Priv = 1
LD.CC = 1
34
R6 ← R6 + 2
[PSR[15]]
DR_SPMUX = 1
SR1_SPMUX = 1
SPMUX = 2
GateSP = 1
LD.Reg = 1
COND4
PSR[15] == 0
PSR[15] == 1
57
both start
18
54
SSP ← R6
R6 ← USP
18
LD.Reg
DR_SPMUX = 1
SR1_SPMUX = 1
LD.Saved_SSP
SPMUX = 0
GateSP
INT:
49
MDR ← PSR
PSR[15] ← 0
[PSR[15]]
GatePSR = 1
LDMDR = 1
LD.Priv = 1
COND = 4
Data.Size = 1
PSR[15] == 0
PSR[15] == 1
45
USR ← R6
R6 ← SSP
LD.REG
LD.SavedUSP
GateSP
SPMUX = 1
37
Saved_next_state ← microinstruction.J
R6, MAR ← R6 − 2
Set MV
GateSP
SPMUX = 3
LD.MAR
DR_SPMUX
SR1_SPMUX
DR_SPMUX = 1
SR1_SPMUX = 1
LD.REG
PSR → MDR
41
M[MAR] ← MDR
R
copy 16
43
Saved_next_state ← microinstruction.J
R6, MAR ← R6 − 2
Set MV
LD.MAR
LD.REG
SR1_SPMUX
GateSP
SPMUX = 3
DR_SPMUX
46
MDR ← PC − 2
GatePC = 1
Data.Size = 1
LD.MDR
copy 16
52
m[MAR] ← MDR
R
54
Saved_next_state ← microinstruction.J ; Set MV
MAR ← x0200 + LSHF(vector, 1)
GateVector
LD.MAR
53
MDR ← m[MAR]
R
copy 25
55
PC ← MDR
LD.PC
GateMDR
PCMUX = 1
Data.size = 1
18

When loading into MAR, save next state, and check for unaligned
↳ this is updated in the original FSM up top

if LD.MAR == 1 : # check in microsequencer, except for states below



AV = access validation
↳ checks for both protection exception and page fault

New control signals for lab 5

- GateMAR - put stuff in MAR onto the BUS
- LDMARMUX - select between putting BUS into MAR or the concatenation of PFN and offset bits into the MAR
- TranslationMUX, GateTranslation - selects between putting VA, VA left shifted by 1 plus PTBR, or PTBR on the BUS
- LD_VA - load enable for VA from BUS
- LD PTBR = load enable for PTBR from BUS
- SNS MUX, LDSNS - selects between putting the j bits into the Saved_Next_state register or clearing the register
- LD AV - uses logic to determine if the access violation should be set or not set
- MDRMUX - selects between loading into MDR normally, or setting either bit 1 or bit 0 of the MDR
- Chk.SNS - help determine the next state when evaluating the microsequencer
- Chk.AV - help determine the next state when evaluating the microsequencer

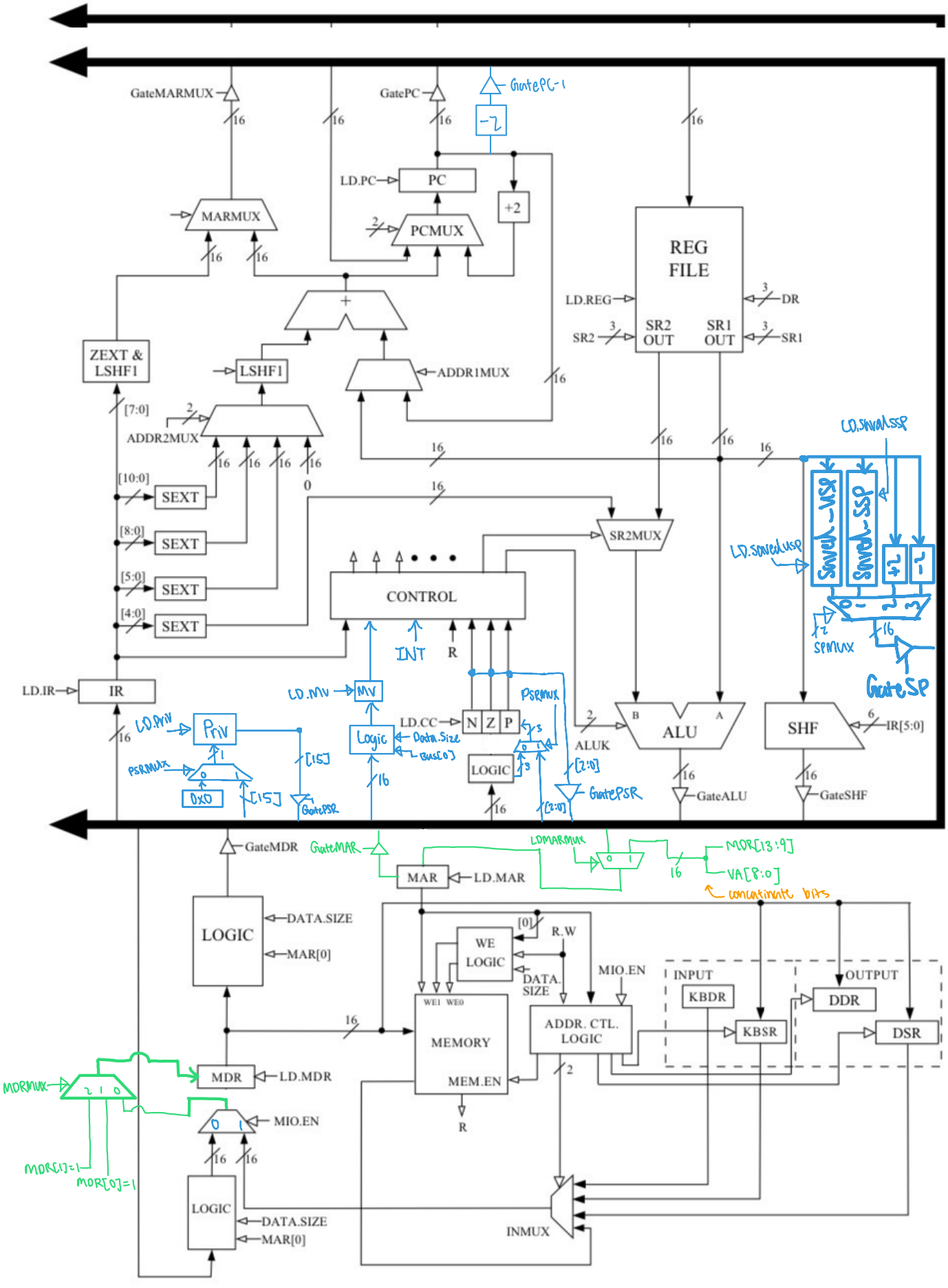
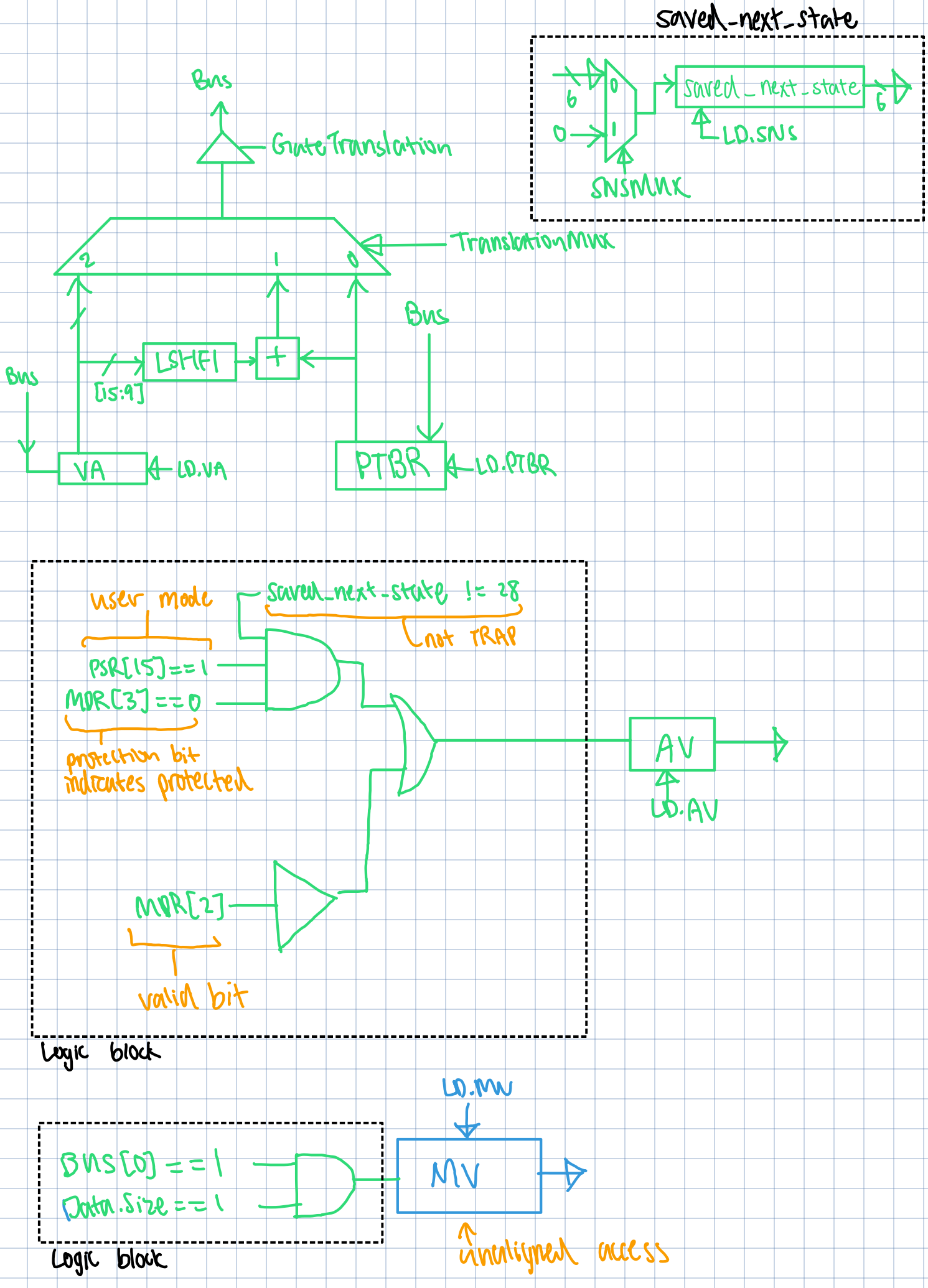
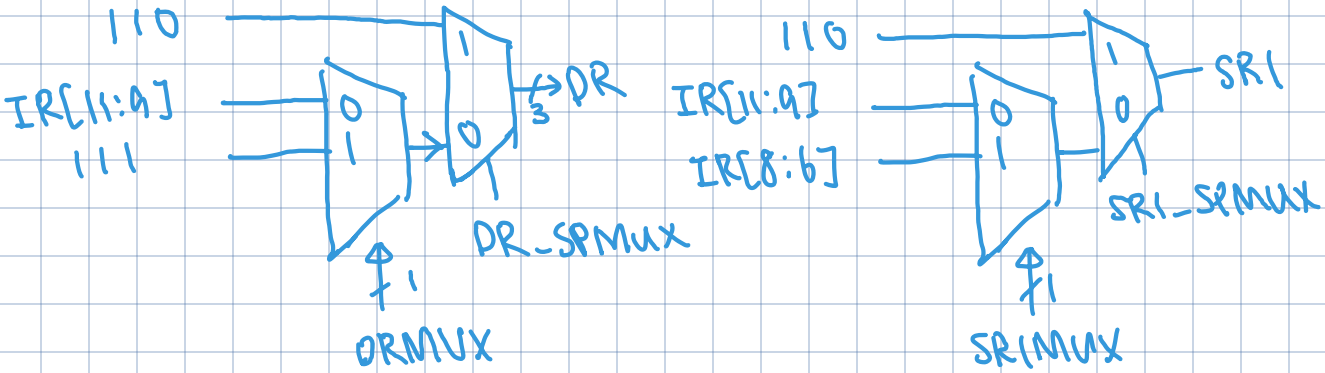
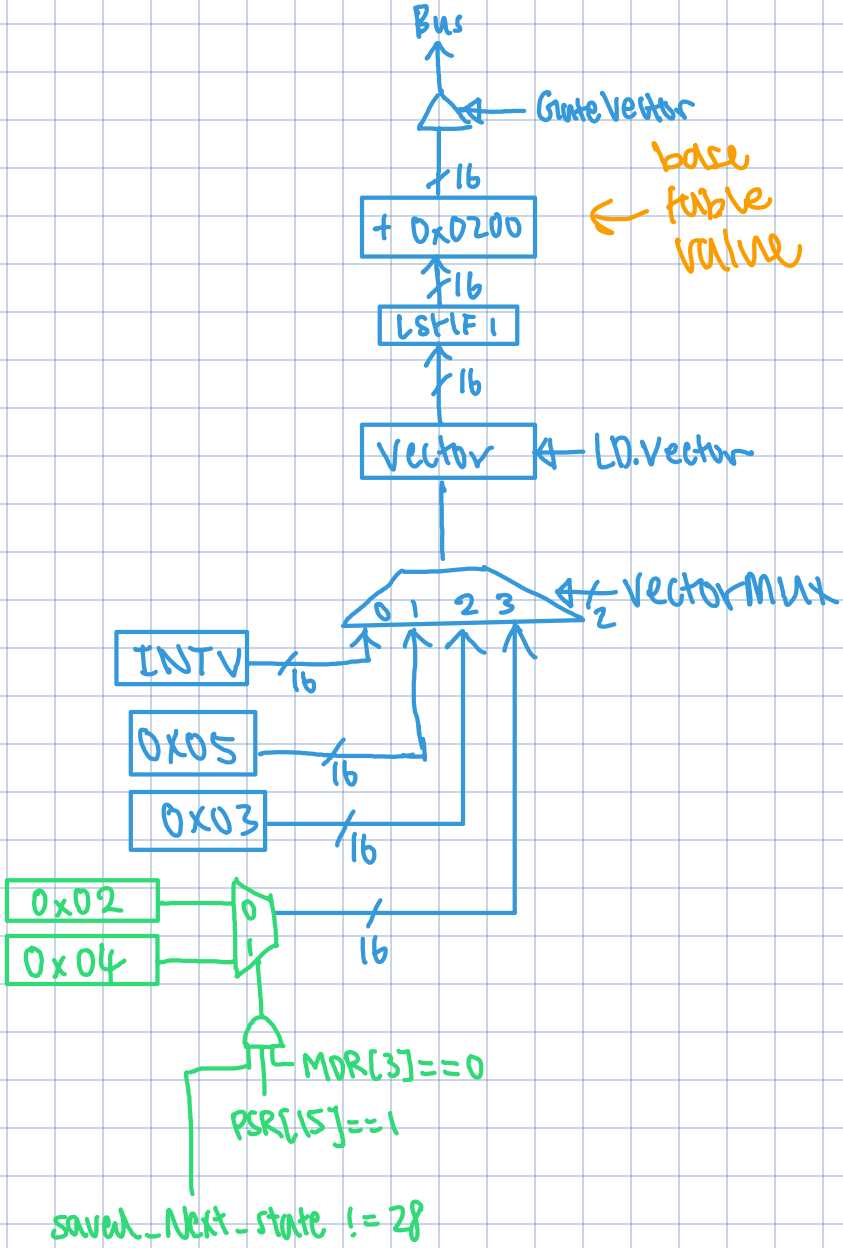
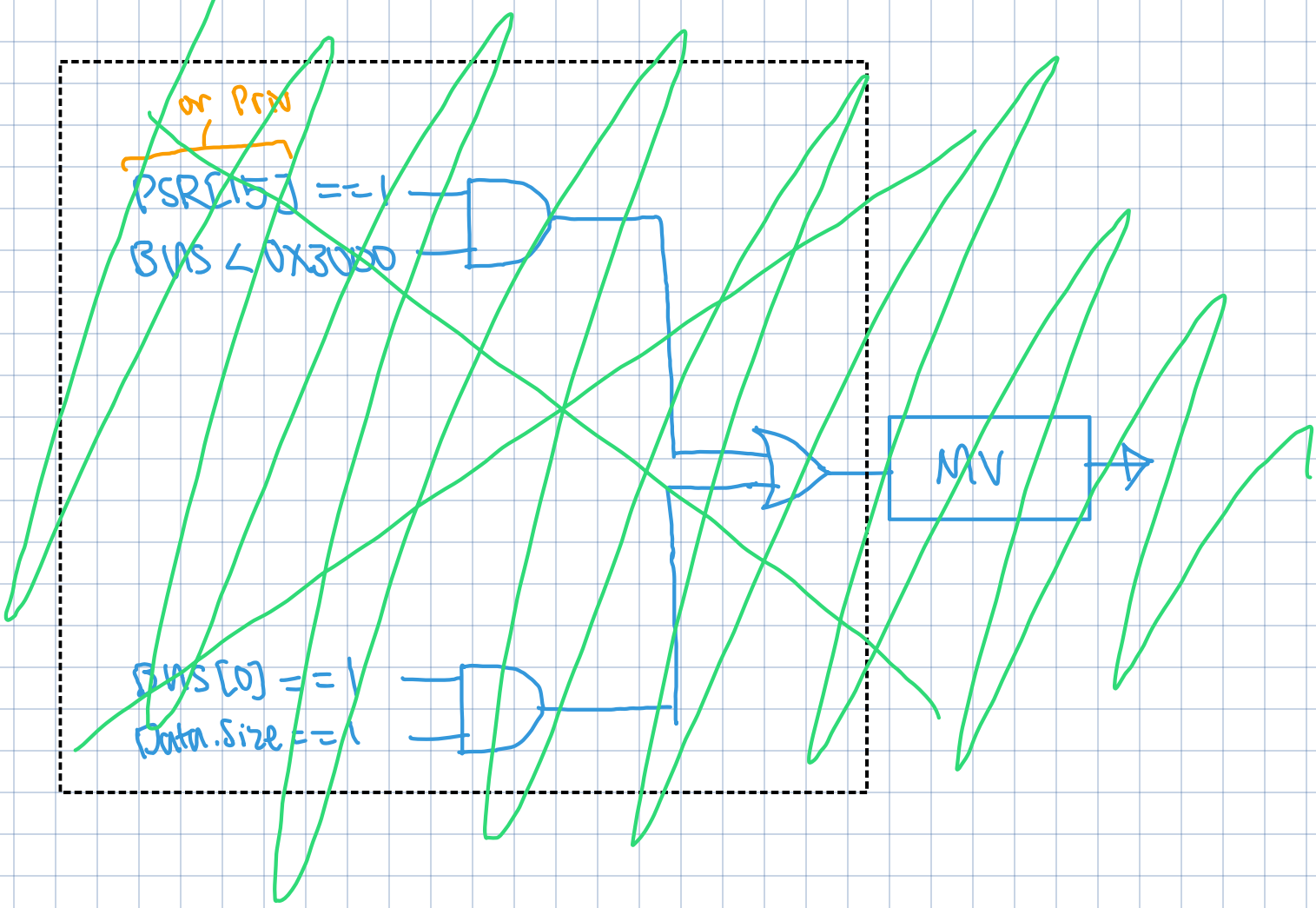


Figure C.3: The LC-3b data path


saved_next_state
6
0
0
1
saved_next_state
6
LD.SNS
SNSMUX
Bus
GateTranslation
TranslationMux
2
1
0
Bus
LSHIFT
+
[15:9]
Bus
VA
LD.VA
PTBR
LD.PTBR
user mode
saved_next_state != 28
not TRAP
PSR[15]==1
MDR[3]==0
protection bit
indicates protected
AV
LD.AV
MDR[2]
valid bit
Logic block
LD.MV
BUS[0]==1
Data.Size==1
MV
unaligned access
Logic block

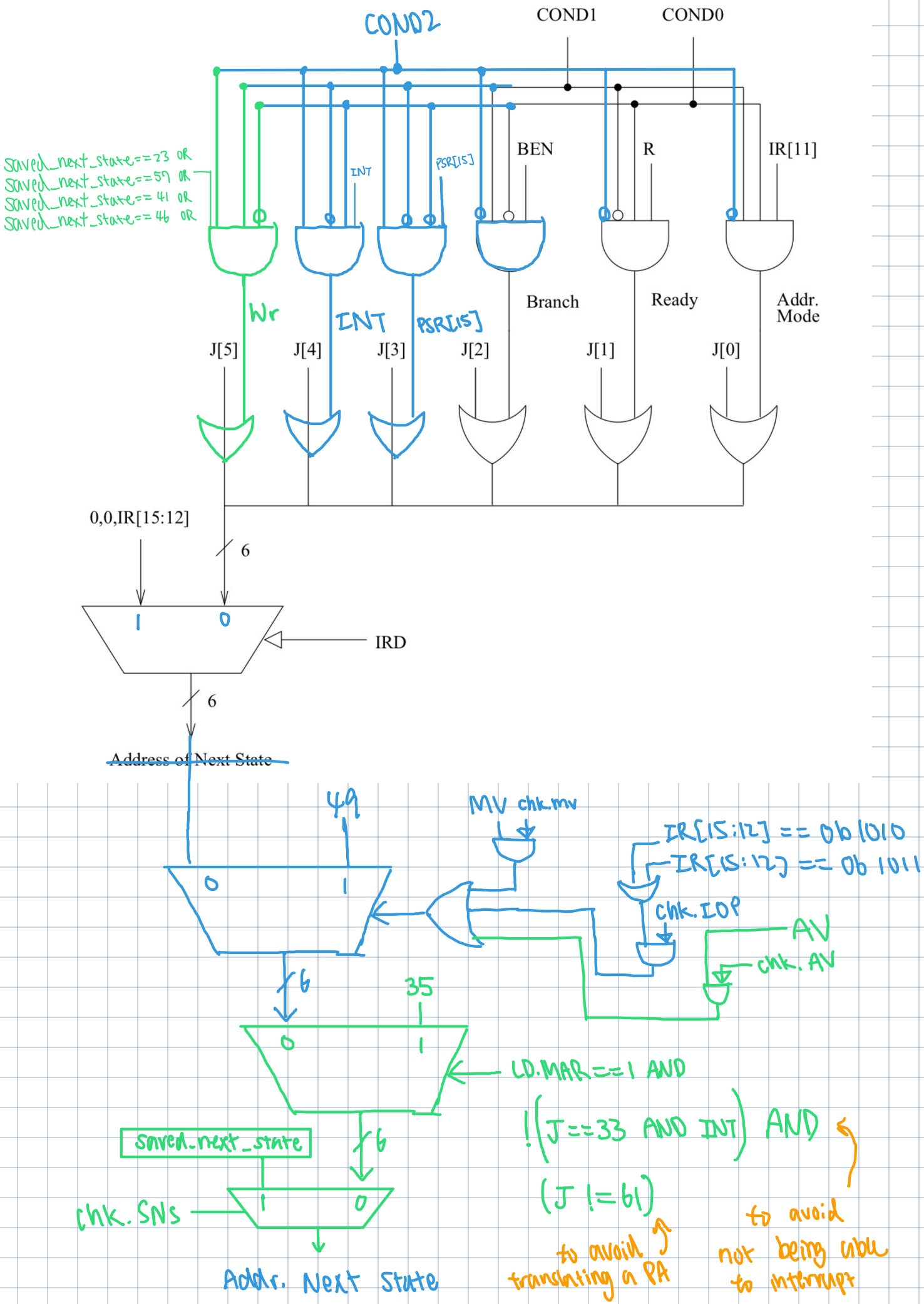

110
IR[11:9]
111
DR
DR_SPMUX
1
ORMUX
110
IR[11:9]
IR[8:6]
SR1
SR1_SPMUX
1
SR1MUX


Bus
GateVector
16
+ 0x0200
base table value
16
LSHF1
16
Vector
LD.Vector
0 1 2 3
2
VectorMUX
INTV
16
0x05
16
0x03
16
0x02
0x04
16
MDR[3]==0
PSR[15]==1
saved_Next_state != 28



Data Path Explanations for Lab 5 New Additions (marked in green in above pages)

- MAR:
  - Added GateMAR for allowing MAR to store into VA register
  - Added more options for loading things into MAR either from BUS or the concatenation of the PFN from the PTE and the offset bits
- MDR:
  - Added new options to set bit 1 or bit 0 of the MDR in order to change the referenced and modified bits of the PTE
- Translation
  - Added options to select between putting VA on the bus, PTBR on the bus, or PTBR + VA left shifted by 1 on the BUS. This allows for either finding the appropriate PTE's PA or to help concatenation the lower offset bits of the VA with the PFN to get the physical address.
- Saved_Next_State register
  - Saves the next state of a state by taking the j field from the micro instruction of the state that takes place before the translation. This allows for translation to happen seamlessly and allow states to go back to their original sequence.
  - Also can clear the register
- Access violation exception checking
  - Logic helps check for either protected access from user mode exception, or for page faults.
  - 1 in AV means that there's an exception. 0 means there's no exception.
- Made MV (memory violation) only check for unaligned access in the virtual address, and removed the check for protected access (because the mechanism for checking protection is now different in lab 5.
- Vector register loading: accommodate one more vector address, and changed some vector addresses due to differing lab specifications. Implemented priority of protected exception over page fault with hardware logic here.


COND2
COND1
COND0
BEN
R
IR[11]
INT
PSR[15]
saved_next_state==23 OR
saved_next_state==59 OR
saved_next_state==41 OR
saved_next_state==46 OR
Wr
INT
PSR[15]
Branch
Ready
Addr. Mode
J[5]
J[4]
J[3]
J[2]
J[1]
J[0]
0,0,IR[15:12]
6
1
0
IRD
6
Address of Next State
49
MV chk.mv
IR[15:12] == 0b1010
IR[15:12] == 0b1011
chk.IOP
AV
chk.AV
0
1
6
35
0
1
LD.MAR==1 AND
!(J==33 AND INT) AND
(J!=61)
to avoid translating a PA
to avoid not being able to interrupt
saved.next_state
6
chk.SNS
1
0
Addr. Next State

Microsequencer changes

- Added one last cond option in order to allow for branching to determine whether the modified bit of the PTE should be set to 1. This is determined by seeing if the next state of the instruction that is going under translation is an instruction that writes to memory. These states are hardcoded.
- Added one more chk.av to accommodate for the more exceptions specifications of lab 5. This just means that there's one more scenario in which the program ends up at state 49 for a context switch for exception — when there's page fault.
- Added one more MUX for selecting state 35 when VA is loaded into the MAR, except for when there's an interrupt, or during the translation process itself (to avoid infinite loop).
- Added one more MUX to utilize the Saved_Next_state register to return back to the appropriate state after translation.

Signal Name	Signal Values	
J/6:		
COND/~~2~~ 3:	$COND_0$	;Unconditional
	$COND_1$	;Memory Ready
	$COND_2$	;Branch
	$COND_3$	;Addressing Mode
CHK.MV / 1 CHK.IOP/1 IRD/1:	NO, YES	

$COND_4$: PSR[15]
$COND_5$: INT

$COND_6$: Wr

Table C.2: Microsequencer control signals